Process Synchronization
Part 2

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Overview

- Deadlocks
- Coarse vs Fine-grained locking
- Lock ordering
- Livelocks

Recall from last lecture

- **Processes often share data**

- **A race condition exists when access to shared data is not controlled**
  - Possibly resulting in corrupt data values and nasty bugs – hard to discover, hard to reproduce, hard to debug

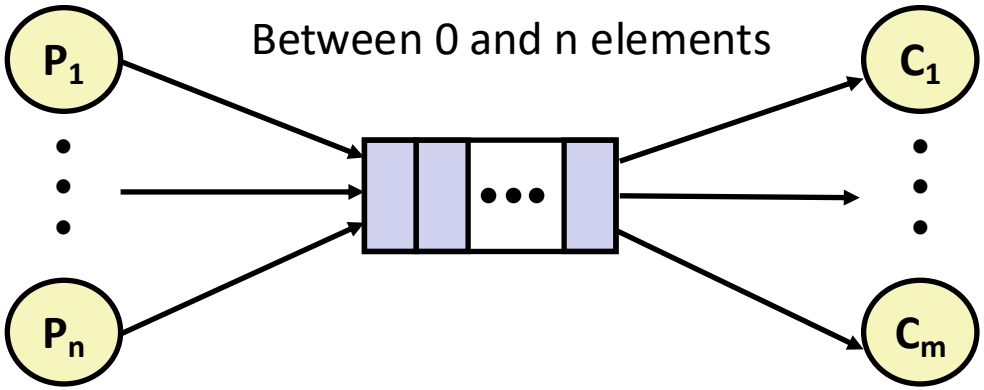
- **Process synchronization involves using tools that control access to shared data to avoid race conditions**
  - Semaphores are a versatile synchronization mechanism that enables two or more processes to coordinate their actions

Recall from last lecture

- **Under the normal mode of operation, a process may utilize a shared resource in only the following sequence:**
  1. **Request**
     - If the request cannot be granted immediately, then the requesting process must wait until it can acquire the resource
  2. **Use (critical section)**
     - The process can operate on the resource (data integrity must be ensured)
  3. **Release**
     - The process releases the resource

Review: Producer-Consumer problem

- **Requires mutual exclusion and two counting semaphores**
  - `mutex`: enforces mutually exclusive access to the buffer and counters
  - `slots`: counts the available slots in the buffer
  - `items`: counts the available items in the buffer



Safety and liveness

- **Now is a good time to pop up a level and look at what we're doing**
- **Our primary goals should be to create software that is safe from bugs, easy to understand, and ready for change**
  - Building concurrent software is clearly a challenge for all three of these goals
- **When we ask whether a concurrent program is safe from bugs, we care about two properties: safety and liveness**

Safety and liveness

- **Safety – can you prove that some bad thing never happens?**
  - Does the concurrent program satisfy its invariants and its specifications?
  - **Races in accessing mutable data threaten safety**

- **Liveness – can you prove that some good thing eventually happens?**
  - Does the program keep running and eventually do what you want, or does it get stuck somewhere waiting forever for events that will never happen?
  - **Deadlocks and livelocks threaten liveness**

Safety and liveness

- **Ensuring liveness may also require fairness**
  - Fairness means that concurrent processes receive sufficient CPU time to make progress in their execution

- **Fairness is primarily enforced by the OS scheduler**
  - However, applications can influence scheduling behavior by adjusting process priorities
  - (more on this in future lectures)

Today's discussion

- **When used properly and carefully, synchronization tools can prevent race conditions**

- **However, another set of problems arises with their misuse:**
  - Degraded system performance
  - Liveness hazards, including deadlocks and livelocks

- **Let's introduce a small change in our code to address these concerns**
  - This version **preserves safety but violates liveness**

Initialization and clearing

```
/* Initializes semaphores */
mutex = sem_open("/mutex", O_CREAT, 0644, 1);
slots = sem_open("/slots", O_CREAT, 0644, N);
items = sem_open("/items", O_CREAT, 0644, 0);

/* Create an empty, bounded, shared FIFO buffer with N slots */
void sbuf_init(sbuf_t *sp){
  memset(sp->buffer, 0, N*sizeof(int));
  sp->in = sp->out = 0;
}
```

Producer

```
/* Insert item onto the rear of shared buffer sp */
void sbuf_insert(sbuf_t *sp, int item){

   /* Lock the buffer */
   sem_wait(mutex);          /* moved here */

   /* Wait for available slot */
   sem_wait(slots);

   /* store in buffer */
   sp->buffer[sp->in] = item;
   sp->in = (sp->in + 1) % N;

   /* Announce available item */
   sem_post(items);

   /* Unlock the buffer */
   sem_post(mutex);          /* moved here */
}
```

Critical section

Consumer

```
/* Remove and return the first item from buffer sp */
int sbuf_remove(sbuf_t *sp){
  int item;

  /* Lock the buffer */
  sem_wait(mutex);          /* moved here */

  /* Wait for available item */
  sem_wait(items);

  /* load it from buffer */
  item = sp->buffer[sp->out];
  /* consume the item */
  sp->out = (sp->out + 1) % N;

  /* Announce available slot */
  sem_post(slots);

  /* Unlock the buffer */
  sem_post(mutex);          /* moved here */
  return item;
}
```

Critical section

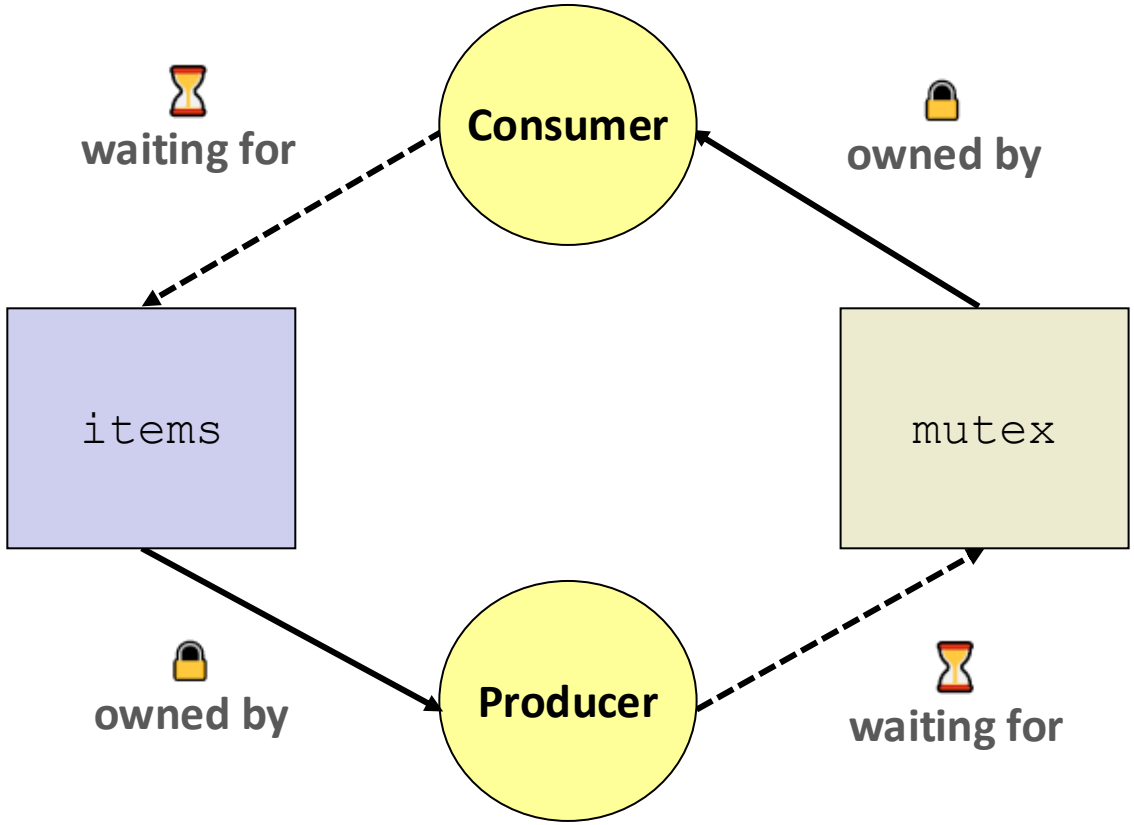
Deadlock

- **The program is no longer deadlock-free**
  - Its liveness now depends on the specific scheduling sequence of producers and consumers

- **Consider the following sequence of events:**
  1. Consumer runs and can decrease `mutex` – it now **holds the semaphore**
  2. It now **blocks** as it tries to decrease `items` – there is no elements in buffer
  3. Producer runs but it **cannot decrease** `mutex` – it's held by the consumer
  4. It **blocks**, preventing all processes to proceed

- **This new version of our code can lead to a deadlock state**
  - Causing all processes to stall and preventing any further progress

Deadlock


waiting for
Consumer
owned by
items
mutex
owned by
Producer
waiting for

Necessary conditions

- **This example illustrates a problem with handling deadlocks**
  - It is difficult to identify and test for deadlocks that may occur only under certain scheduling circumstances
  - Static and dynamic analysis tools often struggle to accurately detect deadlocks in code with dynamic locking behaviors

- **A deadlock situation can arise if the following four conditions hold simultaneously in a system:**
  1. Mutual exclusion
  2. Hold and wait
  3. No preemption
  4. Circular wait

Necessary conditions

- **Mutual exclusion**
  - At least one resource must be **held in a non-sharable mode**
  - If another process requests that resource, the requesting process must be delayed until the resource has been released

- **Hold and wait**
  - A process must be **holding at least one** resource and **waiting to acquire additional resources** that are currently being held by other processes

Necessary conditions

- **No preemption**
  - A resource can be released only **voluntarily by the process holding it**, after that process has completed its task

- **Circular wait**
  - A **set** $\{P_0, P_1, \ldots, P_n\}$ **of waiting processes** must exist such that P_0 is waiting for a resource held by P_1, P_1 is waiting for a resource held by P_2, ..., P_{n-1} is waiting for a resource held by P_n, and P_n is waiting for a resource held by P_0

Handling deadlocks

- **Deadlock prevention**
  - Strict policies to avoid deadlocks by design, often at the expense of concurrency
  - The system provides a set of methods to ensure that at least one of the necessary conditions cannot hold
  - Usually focuses on breaking either the “no preemption” or the “circular wait” requirement of deadlock (most common)

- **Deadlock avoidance**
  - More dynamic and flexible resource allocation by continuously ensuring the system remains in a safe state
  - The system evaluates each request and grants it only if the resulting state is safe
  - E.g., Banker’s algorithm, wound/wait, and wait/die algorithms

OS methods for handling deadlocks

1. **Ensure that the system will never enter a deadlock state**
   - Certain specialized OS kernels or subsystems (e.g., HPC, real-time systems, or embedded systems) implement formal deadlock avoidance to guarantee high reliability
   - Deadlocks can be modeled with resource-allocation graphs
   - In systems with a single instance of each resource, a cycle implies deadlock

2. **Allow the system to enter a deadlock state and then recover**
   - If a system does not employ either a deadlock-prevention or a deadlock-avoidance algorithm, then a deadlock situation may occur
   - In this environment, the system provides an algorithm to recover from the deadlock
   - Hardly used in OSes; used heavily in database systems

OS methods for handling deadlocks

3. Ignoring the possibility of deadlocks

- Assume deadlocks are rare and do nothing
- Widely used in most end-user OSes (lower overhead)
- Responsibility falls on developers to avoid deadlocks, typically using approaches outlined for deadlock prevention and avoidance

Programmer deadlock prevention techniques

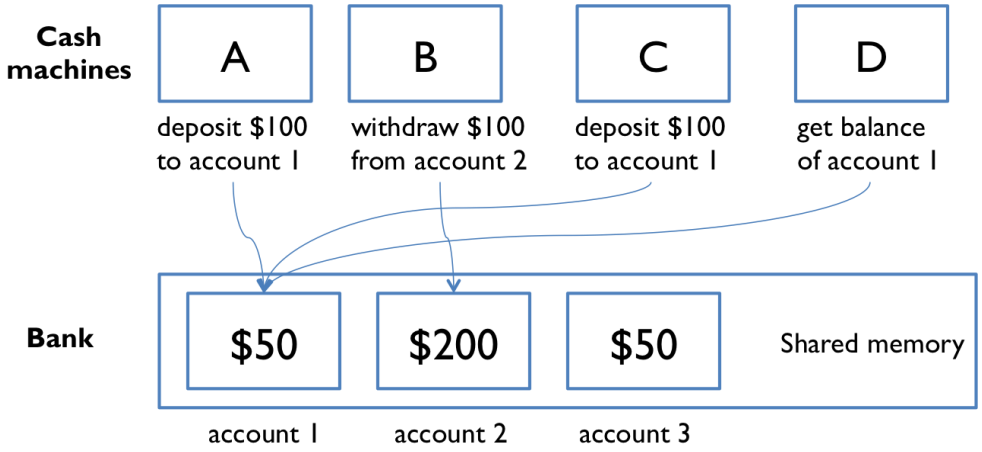
- **Design simple solutions**
  - Complex synchronization mechanisms invite deadlocks
- **Do not attempt to acquire a semaphore that is already held by the same process**
  - Reentrant variants exist but cannot be used as an event signaling mechanism
- **Avoid holding several semaphores**
  - Consider using a single semaphore to the set of shared resources in scenarios with low concurrency and contention

Overview

- Deadlocks
- **Coarse vs Fine-grained locking**
- Lock ordering
- Livelocks

Example: Bank system

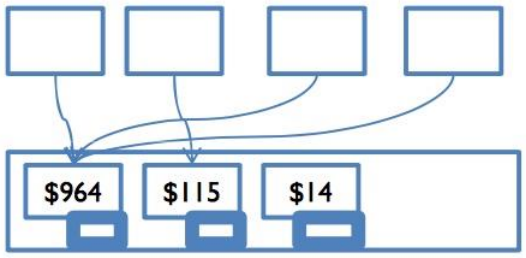
- **Imagine that a bank has cash machines that use a shared memory model**
  - So, all the cash machines can read and write the same account objects in memory



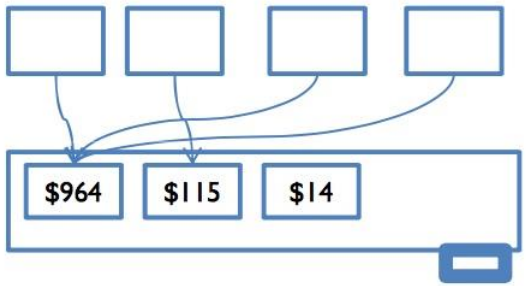
- **Without any coordination between concurrent reads and writes to the account balances, things can go horribly wrong**

Coarse vs Fine-grained locking

- Deciding on the number of needed semaphores for a correct process synchronization can be a difficult exercise
- Coarse-grained – few locks, each protecting lots of data
- Fine-grained – many locks, each protecting little data
- In our bank system, two extreme cases:



one lock per account



one lock for the whole bank

Coarse-grained locking

+ Easier to write and to prove correct

- For many applications, this is the best solution
- Don’t underrate simplicity!

– Poor performance when contention is high

- Essentially, no concurrent access (if using only one lock)
- Adding more processes does not improve throughput

Coarse-grained locking

- **OSes were originally designed for single-processor machines**
  - Disabling interrupts was sufficient for the kernel to prevent race conditions on kernel state
- **When Linux first added multicore support in 1996, it used a Big Kernel Lock that was acquired at the start of any context switch into the kernel**
  - Advantage: Simple, and still allowed user-level code to be parallel
  - Disadvantage: Only one core can execute kernel code at a time
  - Note: BKL was removed completely in Linux 2.6.39 (2011)
- **Over many years, Linux (and other OSes) developed much finer-grained locking strategies**

Fine-grained locking

+ Improved concurrency and scalability

- More simultaneous access
- Performance increase when coarse-grained would lead to unnecessary blocking

– Higher overhead from having many locks

- Blocked processes are moved to a waiting queue and context-switched
- Resulting in increased CPU consumption and potential performance degradation

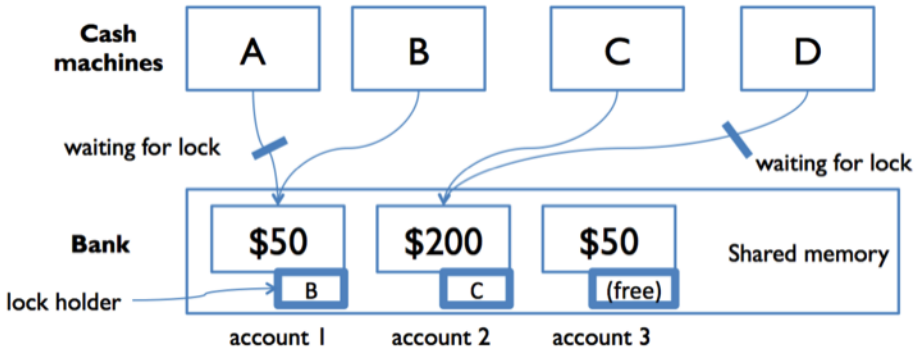
– Requires careful thought

- Correctness is harder to achieve and prove
- Opportunities abound for errors, sometimes subtle, that can cause data races or deadlocks

Example: Fine-grained locking

- **We can add a lock that protects each bank account**
  - Now, before they can access or update an account balance, cash machines must first acquire the lock on that account

- **This ensures that A and B are synchronized, but another cash machine C is able to run independently on a different account**



Deadlock prevention

- **The fundamental challenge in acquiring multiple locks is to prevent deadlocks from occurring**

- **As we saw, for a deadlock to occur, each of the four necessary conditions must hold**
  - By ensuring that at least one of these conditions cannot hold, we can prevent the occurrence of a deadlock

- **One way to prevent deadlock is to break the "circular wait" condition**
  - Impose an ordering on the locks that need to be acquired simultaneously, and ensuring that all code acquires the locks in that order

Overview

- Deadlocks
- Coarse vs Fine-grained locking
- **Lock ordering**
- Livelocks

Lock ordering

- **We can accomplish this scheme in an application program by developing an ordering among all resources in the system**
  - E.g., assign a numerical integer value to all resources
- **We can now consider the following protocol:**
  - Each process can request resources only in an **increasing order of enumeration**
  - If a process acquires a later resource in the ordering, then wants an earlier resource in the ordering, **must release all its locks and start over**
  - Therefore, there can be no circular wait
- **However, establishing a lock ordering can be difficult**
  - Particularly on a system with hundreds – or thousands – of locks

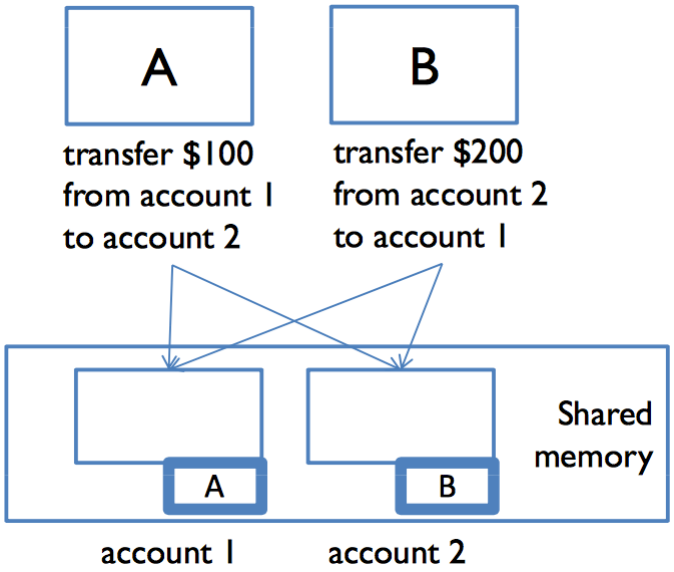
Example: Lock ordering

- **Assume we have a function that transfers funds between two accounts**
  - To prevent a race condition, each account has an associated mutual exclusion semaphore
  - To try to prevent a deadlock, first acquire the mutex of `fromAccount`, then the mutex of `toAccount`

```
void transferMoney(Account *fromAccount, Account *toAccount,
                   float amountToTransfer){
  sem_wait(fromAccount->mutex);
  sem_wait(toAccount->mutex);
  debit(fromAccount,amountToTransfer);
  credit(toAccount,amountToTransfer);
  sem_post(toAccount->mutex);
  sem_post(fromAccount->mutex);
}
```

Lock ordering

- **Deadlock is still possible if two processes simultaneously invoke the `transferMoney()` function, inverting different accounts**
  - `transferMoney(A, B, 10)` locks A then B
  - `transferMoney(B, A, 30)` locks B then A
  - This violates the circular-wait prevention strategy



Lock ordering

- **Lock ordering must be global and consistent**
  - E.g., enforce a global lock ordering based on account ID

```
void transferMoney(Account *fromAccount, Account *toAccount,
                   float amountToTransfer){
    Account *first, *second;

    if(fromAccount->id < toAccount->id
        first  = fromAccount;
        second = toAccount;
    }else{
        first  = toAccount;
        second = fromAccount;
    }

    /* Acquire locks in global order */
    sem_wait(&first->mutex);
    sem_wait(&second->mutex);
    debit(fromAccount, amountToTransfer);
    credit(toAccount, amountToTransfer);
    sem_post(&second->mutex);
    sem_post(&first->mutex);
}
```

Overview

- Deadlocks
- Coarse vs Fine-grained locking
- Lock ordering
- **Livelocks**

Non-blocking variants

- **In complex systems, a strict global ordering of resources may not be possible**
  - The system may not have a clear hierarchy
  - Resources may be created dynamically
  - Relationships between resources may change over time

- **In these situations, deadlock prevention can rely on non-blocking lock attempts**

- **Instead of waiting indefinitely for a lock, the process can:**
  1. Attempt to acquire the lock
  2. Abort or retry later if the lock is unavailable

Non-blocking variants

- **POSIX provides mechanisms for this approach:**
  - `sem_trywait()` – immediately returns if the semaphore cannot be acquired
  - `sem_timedwait()` – waits only for a bounded amount of time

- **This strategy allows the program to detect potential deadlocks and retry safely**

Non-blocking variants

- **Releasing held resources when a conflict occurs prevents the hold-and-wait condition**
  - Eliminating one of the necessary conditions for deadlock

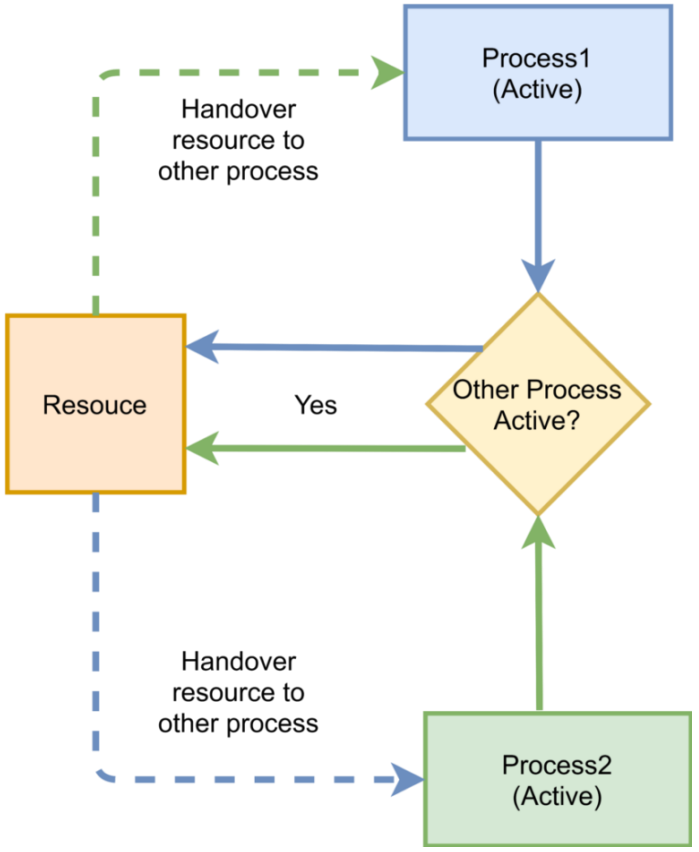
```
void transferMoney(Account *fromAccount, Account *toAccount,
                   float amountToTransfer){
  int done = 0;

  while(!done){
    sem_wait(fromAccount->mutex);
    if(sem_trywait(toAccount->mutex) == 0){
      debit(fromAccount,amountToTransfer);
      credit(toAccount,amountToTransfer);
      sem_post(toAccount->mutex);
      sem_post(fromAccount->mutex);
      done = 1;
    }else{
       sem_post(fromAccount->mutex);
    }
  }
}
```

Overview

- Deadlocks
- Coarse vs Fine-grained locking
- Lock ordering
- **Livelocks**

Livelock

- **Previous solution can lead to a livelock**
  - Occurs when a process continuously attempts an action that fails



Livelock

- **It is like a deadlock in the sense that both prevent two or more processes from proceeding**
  - But the processes are unable to proceed for different reasons
- **Typically occurs when processes retry failing operations at the same time**
  - It thus can generally be avoided by having each process retry the failing operation at **random times**
- **Less common than deadlock but nonetheless is a challenging issue in designing concurrent applications**
  - Like deadlock, it may only occur under specific scheduling circumstances

Handling livelocks

```
void transferMoney(Account *fromAccount, Account *toAccount,
                   float amountToTransfer){
  int done = 0;
  int random_ms;

  while(!done){
    sem_wait(fromAccount->mutex);
    if(sem_trywait(toAccount->mutex) == 0){
      debit(fromAccount,amountToTransfer);
      credit(toAccount,amountToTransfer);
      sem_post(toAccount->mutex);
      sem_post(fromAccount->mutex);
      done = 1;
    }else{
       sem_post(fromAccount->mutex);
    }
    random_ms = rand() % 1000;
    usleep(random_ms * 1000);
  }
}
```

* Using `rand()` without seeding may produce identical patterns in multiple processes

Summary

- **Correct concurrent programs must guarantee:**
  - Safety (nothing bad happens)
  - Liveness (something good eventually happens)

- **Liveness issues include:**
  - Deadlock
  - Livelock
  - Starvation (next lecture)

- **Deadlock occurs when processes are permanently blocked waiting for each other**
  - Four necessary conditions for deadlock: mutual exclusion, hold and wait, no preemption, circular wait

Summary

- **Deadlock can be handled by:**
  - Prevention (break one condition)
  - Avoidance (stay in safe states)
  - Detection and recovery

- **Lock granularity impacts both safety and performance**
  - Coarse-grained – simpler, but lower concurrency
  - Fine-grained – higher concurrency, but greater complexity